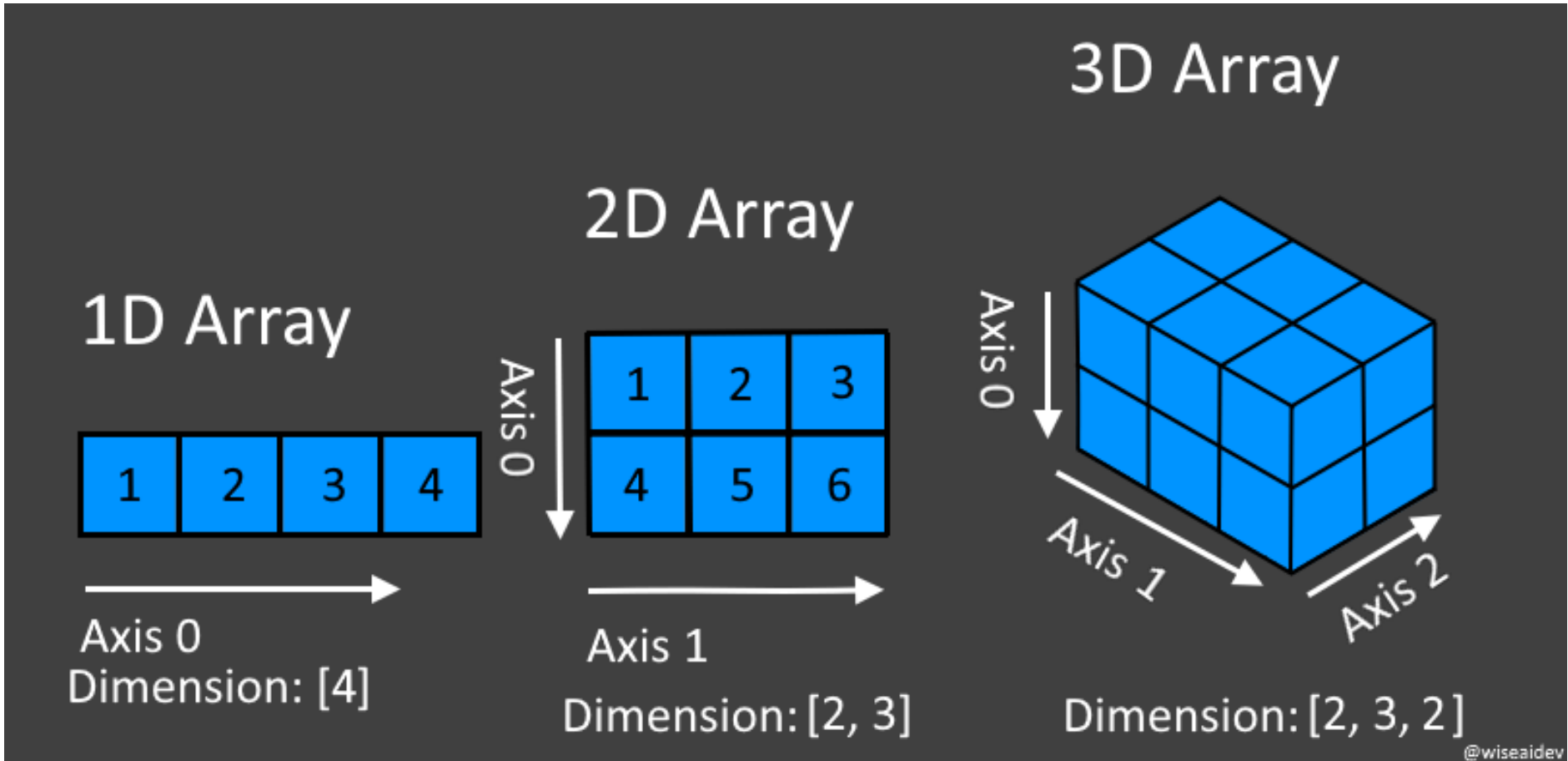


Data representations for neural networks

Data representations for neural networks

- ML ในปัจจุบันจะใช้ **tensor** เป็นโครงสร้างของข้อมูลพื้นฐาน
- **Tensor** คือ ตัวเก็บข้อมูลหรือที่เก็บข้อมูล ส่วนใหญ่จะเป็นข้อมูลชนิดตัวเลข
- ถ้าเป็น **tensor** ไม่มีมิติ → เรียกว่า **Scalar**
- ถ้าเป็น **tensor** มิติเดียว → เรียกว่า **Vector**
- ถ้าเป็น **tensor** สองมิติ → เรียกว่า **Matrix**
- ถ้าเป็น **tensor** มากกว่า 2 มิติ → เรียกว่า **k-th order tensor**
- **Tensor** ถูกมองว่าเป็นการขยายแนวคิดของ **vector** และ **matrix** ให้ครอบคลุมมิติใดๆ

Tensors



Tensors

- deep learning มักจะใช้ tensor ตั้งแต่ 0 มิติ ถึง 4 มิติแต่สำหรับข้อมูลวิดีโอ อาจต้องใช้เทนเซอร์ถึง 5 มิติ

- 1D

$$[\text{feature}_1, f_2, \dots, f_n]$$

- 2D ใช้งานกับ ตารางข้อมูลต่างๆ

$$[[f_1, f_2, f_3]_{\text{sample}1}, [f_1, f_2, f_3]_{s2}, \dots, [f_1, f_2, f_3]_{sn}]$$

$$\text{Or } [[f_1, f_2, f_3]_{\text{time}1}, [f_1, f_2, f_3]_{t2}, \dots, [f_1, f_2, f_3]_{tn}]$$

- 3D ใช้งานกับ ราคาหุ้น ข้อความ

$$\begin{bmatrix} [[f_1, f_2, f_3]_{t1}, [f_1, f_2, f_3]_{t2}, \dots, [f_1, f_2, f_3]_{tn},]_{s1}, \\ [[f_1, f_2, f_3]_{t1}, [f_1, f_2, f_3]_{t2}, \dots, [f_1, f_2, f_3]_{tn},]_{s2}, \\ \vdots \\ [[f_1, f_2, f_3]_{t1}, [f_1, f_2, f_3]_{t2}, \dots, [f_1, f_2, f_3]_{tn},]_{sn} \end{bmatrix}$$

Tensors

- 4D (samples, height, width, channels) หรือ (samples, channels, height, width) ใช้งานกับภาพ

$$\bullet \left[\begin{array}{c} \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h1}, \\ \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h2}, \\ \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h3} \end{array} \right]_{s1}, \\ \left[\begin{array}{c} \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h1}, \\ \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h2}, \\ \left[\begin{array}{c} [R_1, G_1, B_1]_{w1}, [R_2, G_2, B_2]_{w2}, [R_n, G_n, B_n]_{w3} \end{array} \right]_{h3} \end{array} \right]_{s2} \end{array} \right]$$

Tensor

- 5D (samples, frames, height, width, channels) หรือ (samples, frames, channels, height, width) ใช้งานกับ video

Scalar (0D tensor)

- Scalar หรือ scalar tensor คือ tensor ที่มีแต่ตัวเลข
- ใน numpy จะใช้ค่าตัวเลขชนิด float32 หรือ float64
- ใช้ .ndim เพื่อดูจำนวนมิติของ tensor ได้
- จำนวนมิติ หรือ แกน ของ tensor เรียกอีกอย่างหนึ่งว่า rank (อันดับของ tensor)

Ex

- 12 เป็น scalar

Code

```
import numpy as np
x = np.array(12)
x, x.ndim
```

{ } Variables			
Filter variables			
Display values in code editor			
While executing			
Name	Type	Shape	Value
x	ndarray	()	array(12)

Vectors (1D tensors)

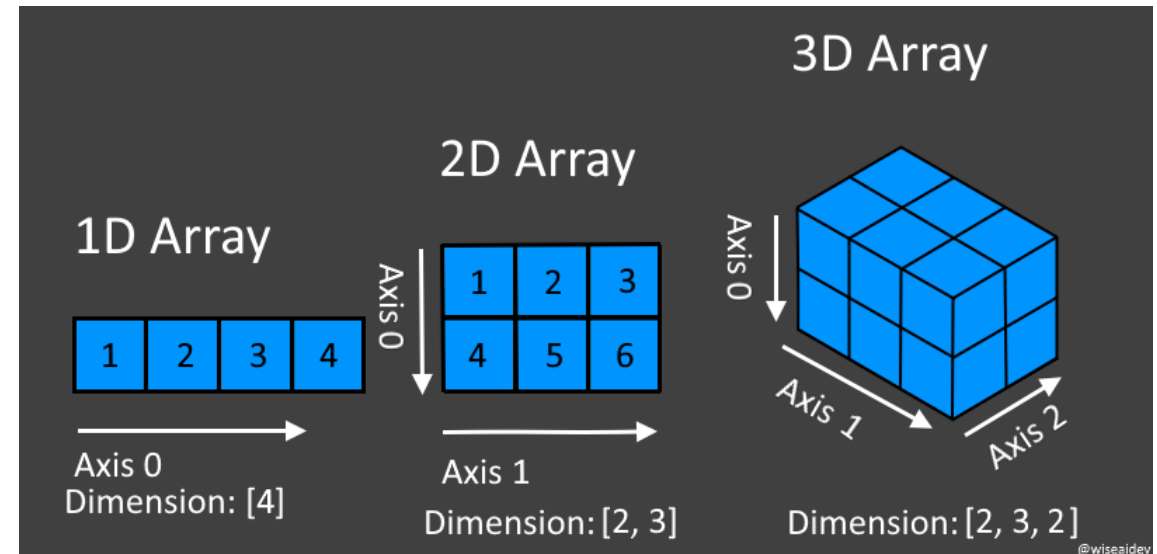
- Vecotor หรือ 1D tensor คือ array ของตัวเลขที่มีเพียบ 1 แกน

Ex

- [12, 3, 6, 14] เป็น vector

Code

```
x = np.array([12, 3, 6, 14])  
x, x.ndim
```



!!!อย่าสับสน!!!

- Vector ที่มีสมาชิก 4 ตัว เรียกว่า เวกเตอร์ 4 มิติ (4D vector) มี 1 แกน (rank1)
- Tensor 4 มิติ (4D tensor) มี 4 แกน (rank 4)

Matrices (2D tensors)

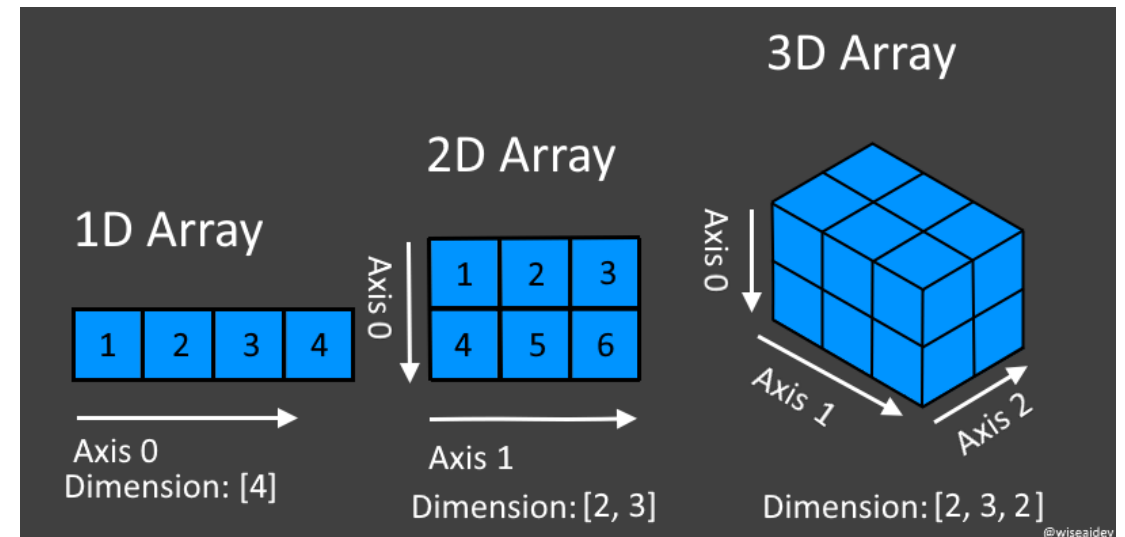
- Matrix หรือ 2D tensor คือ array vector ที่มี 2 แกน (rank 2)
- แกนทั้งสองประกอบด้วย **rows** และ **column**

Ex

• $\begin{bmatrix} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{bmatrix}$ เป็น matrix

Code

```
x = np.array([[5, 78, 2, 34, 0],  
[6, 79, 3, 35, 1],  
[7, 80, 4, 36, 2]])  
x.ndim
```

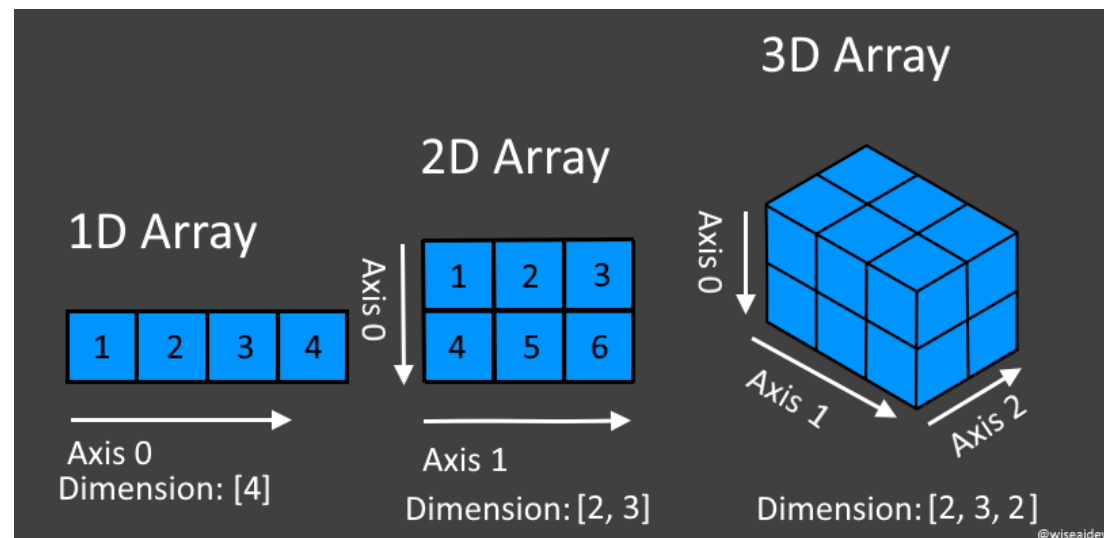


3D tensors and higher-dimensional tensors

- 3D tensors คือ การรวม matrix หลายๆ matrix เข้าไว้ด้วยกัน

Ex 3D tensors

$$\left[\begin{bmatrix} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{bmatrix}, \begin{bmatrix} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{bmatrix}, \begin{bmatrix} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{bmatrix} \right]$$



3D tensors and higher-dimensional tensors

Code

```
x = np.array([ [5, 78, 2, 34, 0],  
               [6, 79, 3, 35, 1],  
               [7, 80, 4, 36, 2] ],  
             [ [5, 78, 2, 34, 0],  
               [6, 79, 3, 35, 1],  
               [7, 80, 4, 36, 2] ],  
             [ [5, 78, 2, 34, 0],  
               [6, 79, 3, 35, 1],  
               [7, 80, 4, 36, 2] ]])
```

```
x.ndim
```

4D tensors

Ex 4D tensor

$$\left[\begin{array}{c} \left[\begin{array}{c} \left[\begin{array}{ccccc} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{array} \end{array} \right], \left[\begin{array}{c} \left[\begin{array}{ccccc} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{array} \end{array} \right], \left[\begin{array}{c} \left[\begin{array}{ccccc} 5 & 78 & 2 & 34 & 0 \\ 6 & 79 & 3 & 35 & 1 \\ 7 & 80 & 4 & 36 & 2 \end{array} \end{array} \right] \end{array} \right]$$

Ex code ข้อข้างบน

Key attributes of tensor

- คุณสมบัติของ **tensor** มี 3 คุณสมบัติ
1. จำนวนแกน **Number of axes** หรือ **rank**
 - 3D tensor มี 3 แกน
 - Matrix มี 2 แกน
 2. รูปร่าง **shape** บอกขนาดของแต่ละแกน
 - Shape ของ matrix ในหน้า 9 คือ (3, 5)
 - Shape ของ matrix ในหน้า 11 คือ (3,3,5)
 - Shape ของ matrix ในหน้า 12 คือ (2,3,3,5)
 3. ชนิดข้อมูล (**Data type**)
 - float32, float64, uint8

Code

```
from keras.datasets import mnist
```

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
train_images.ndim
```

```
train_images.shape
```

```
train_images.dtype
```

Displaying the fourth digit

Code

```
digit = train_images[4]

import matplotlib.pyplot as plt
plt.imshow(digit, cmap=plt.cm.binary)
plt.show()
```

Manipulating tensors in Numpy

Tensor slicing

- Tensor slicing คือการเลือกองค์ประกอบเฉพาะใน tensor

Ex การเลือกภาพที่ 10 ถึง 99 และเก็บใน array ที่มีขนาด (90,28,28)

```
my_slice = train_images[10:100]  
print(my_slice.shape)
```

Ex จงแสดงภาพ `plot train_images[10:20]` และ `my_slice[0:10]`

Tensor slicing

- ใช้ `:` แทนการเลือกทั้งแกน หรือ การเรียกหลายองค์ประกอบ

```
my_slice1 = train_images[10:100, :, :]  
np.array_equal(my_slice, my_slice1)
```

```
my_slice.shape
```

```
my_slice2 = train_images[10:100, 0:28, 0:28]  
np.array_equal(my_slice, my_slice2)
```

```
my_slice2.shape
```

Tensor slicing

```
A = np.arange(5*3*4)
```

```
print('A', A)
```

```
B = A.reshape(5,3,4)
```

```
B[:, 1:, 2:] #สมาชิกมุมขวาล่าง 4 ตัว
```

```
B[:, :2, :2] #สมาชิกมุมซ้ายบน 4 ตัว
```

- เข้าถึงสมาชิกตัวท้ายด้วยเครื่องหมายติดลบ (-)

```
A[-1] #สมาชิกตัวสุดท้าย
```

```
B[-1] #สมาชิกตัวสุดท้าย
```

```
B[:, -1, :]
```

```
B[:, :, -2:]
```

```
B[:, -2:, -2:]
```

```
#แถวสุดท้าย
```

```
#สองคอลัมน์ท้าย
```

```
#มุมขวาล่าง
```

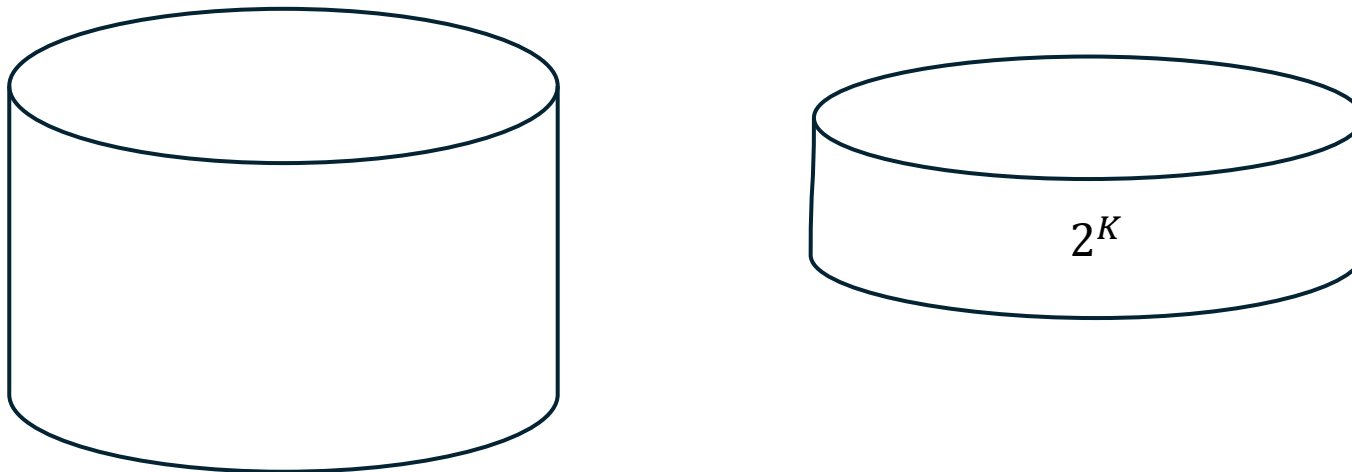
Tensor slicing

```
my_slice3 = train_images[:, 14:, 14:]
```

```
my_slice4 = train_images[:, 7:-7, 7:-7]
```

The notion of data batches

- แนวคิดเรื่องชุดข้อมูลย่อย **data batches** ใน **deep learning** จะไม่ประมวลผลทั้งหมดครั้งเดียว เนื่องจากขนาดข้อมูลอาจจะใหญ่กว่า **memory**
- นิยมเลือกขนาดของ **batch** เป็นเลข 2^k เนื่องจาก **GPU CPU** ถูกออกแบบให้มีประสิทธิภาพในการคำนวณแบบ **block** ในรูปของสองยกกำลัง
- ไม่กำหนดขนาดของ **batch** เล็กเกินไปจะทำให้เกิดความผันผวนสูง
- การแบ่งข้อมูลมักจะแบ่งจาก **axis 0 (sample axis)** หรือ มิติแรก



The notion of data batches

Ex ในตัวอย่างข้อมูล MNIST กำหนดขนาดของ bath size = 128

First batch

```
batch = train_images[:128]
```

Next batch

```
batch = train_images[128:256]
```

n^{th} batch

```
batch = train_images[128*n:128*(n+1)]
```

โจทย์ให้ออกแบบ **tensor** ให้เหมาะสม

1. ข้อมูลคณิตศาสตร์ประกันภัยของข้อมูลบุคคลประกอบด้วย อายุ, รหัสไปรษณีย์ และ รายได้ของแต่ละคน จำนวน 2000 คน
2. ข้อมูลเอกสาร 500 ฉบับ ไม่สนใจลำดับของคำ แต่สนใจจำนวนคำที่ปรากฏ
3. ข้อมูลราคาหุ้นประกอบด้วย ราคาปัจจุบันของหุ้น ราคาสูงสุดในวันที่ก่อนหน้านี้ และ ราคาต่ำสุดในวันที่ก่อนหน้านี้ บันทึกข้อมูลทุกนาที ใช้เวลา 390 นาทีต่อวัน เป็นระยะเวลา 250 วัน
4. ข้อมูลทวิตเตอร์ 1 ล้าน ทวิต สนใจลำดับของคำ แต่ละทวิตมี 280 ตัวอักษร 1 ตัวอักษรเข้ารหัส 128 บิต
5. รูปภาพ 100 รูป สูง 50 กว้าง 20 มี 5 **channels**
6. (samples, frames, height, width, channels) กับ (samples, frames, channels, height, width) ต่างกันอย่างไร